

to improve its submission. Human scores are slow to rise in the initial hours, perhaps as humans spend time digesting the paper.

6. Related Work

In this section, we survey related work for evaluating AI agents on ML research and engineering. For a bigger-picture discussion on how rubric-based evaluation compares to other forms of evaluation and oversight, please see Appendix A.

Evaluating ML Engineering and Research CORE-Bench (Siegel et al., 2024) tasks agents to reproduce the results of a research paper given its repository. In contrast, PaperBench tasks agents to replicate the results of a research paper from scratch.

MLE-bench (Chan et al., 2024), MAgentBench (Huang et al., 2024), and DSBench (Jing et al., 2024) evaluate agents on Kaggle competitions. Many Kaggle competitions are dated and relatively simple ML challenges, whereas PaperBench only contains tasks relevant to modern machine learning research.

RE-Bench (Wijk et al., 2024) proposes 7 challenging open-ended ML research engineering tasks for agents to solve. We expect PaperBench to cover a broader range of sub-tasks over a longer horizon of work compared to the more self-contained tasks proposed in RE-Bench. Additionally, RE-Bench provides agents with a “scoring function” on most tasks to provide a perfect measure of an agents’ performance on the current task; in PaperBench, we are interested in measuring agents’ ability to perform and connect a broad scope of ML research work, where such scoring functions cannot viably capture the full scope of tasks.

Recent work has found that LLMs can generate research ideas of equivalent novelty to human PhDs within specific domains (Si et al., 2024), and solve some toy research problems, involving forming hypotheses, designing and running experiments, and analyzing results (Jansen et al., 2024; Wang et al., 2022).

Automatic judging LLMs have previously been proposed to act as judges to evaluate submissions for tasks (Zheng et al., 2023; Chen et al., 2024; Fu et al., 2023). Agent-based judges have been found to be more accurate than non-agent LLM judges on certain tasks (Zhuge et al., 2024). We benchmark the judging capability of models on significantly harder tasks than what has been used before.

7. Limitations

Dataset Size PaperBench currently consists of only 20 papers, and ideally would capture an even larger portion of the ML research community’s output. However, focusing on the number of papers can be misleading: Since each rubric is composed of hundreds of nodes, PaperBench evaluates agents on thousands of different individual requirements.

Contamination For almost all the papers in our benchmark, the original authors’ codebase for the paper exists online. In our experience, these codebases often do not replicate the entire paper and do not conform to the specific format required for PaperBench submissions (e.g., `reproduce.sh` should exist which executes the code). Nevertheless, models that are pre-trained on large corpuses may have internalized solutions, resulting in inflated performance on this benchmark. While present-day models are most likely not affected by this issue given the recency of the papers in the dataset, this may become an issue for future models.

Challenging dataset creation Producing these detailed rubrics is extremely labor-intensive, each requiring an expert human several full days to create. It requires the creator of the rubric to deeply understand the paper, and each rubric must be carefully written to avoid inaccurate requirements to ensure accurate evaluation. We found it to be challenging to train others to create rubrics at our desired quality level. This poses a challenge for others to replicate the process we undertook to create the dataset. Future work may wish to examine more streamlined approaches to rubric generation, such as with model assistance.

LLM-based judge performance Despite our judge demonstrating good performance in our JudgeEval, it is not as accurate as an expert human judging submissions. Furthermore, our judge is not deterministic due to using non-deterministic model calls. We are excited to see further work in automated judges for complex tasks, as well as future work stress-testing judges via e.g. adversarial submissions. For a broader discussion on complex task evaluation and future advances that will be needed, see Appendix A.

Cost We estimate that on average it costs \$400 in API credits to run an o1 IterativeAgent 12-hour rollout on a single paper in PaperBench. For the 20 papers, this sums to \$8000 USD per eval run. Grading costs an additional \$66 USD per paper on average with o3-mini SimpleJudge. We purposely designed PaperBench Code-Dev (PBCD) not only to eliminate the GPU requirement, but also to address the issue of cost. We expect that PBCD rollouts can be made to run for half the duration of PaperBench roll-outs, due to the lack of execution, which would lead to a cost of \$4000 USD

per eval run. The plateauing observed suggests that the roll-outs may be shortened even further, further reducing costs. We also find that for PBCD, grading costs are reduced to \$10 per paper on average. Finally, we release work on an experimental version of SimpleJudge, with preliminary results showing a 10x decrease in grading costs (See Appendix H).

8. Conclusion

We introduce PaperBench as a challenging benchmark for assessing AI agents’ abilities to replicate cutting-edge machine learning research. Each included paper represents exciting work in a contemporary domain of interest – such as reinforcement learning, robustness, and probabilistic methods – and is evaluated against a rigorous rubric co-developed with the original authors. By requiring AI agents to build entire codebases from scratch, conduct complex experiments, and generate final results, PaperBench offers a demanding real-world test of ML R&D capabilities.

Our experiments with several frontier models suggest that while current AI systems show some capacity to replicate certain facets of machine learning papers, they are still far from competently performing the full range of tasks required for a successful replication. Our strongest evaluated agent in our main setup – Claude 3.5 Sonnet (New) – achieved an average Replication Score of only 21.0%, highlighting both the complexity of ML research tasks and the limitations of current AI agents to conduct complex long-horizon tasks. Nevertheless, these early results underscore non-trivial progress: AI agents succeed in implementing and validating various methods, suggesting promise for future improvements.

By open-sourcing PaperBench, we aim to contribute to evaluating, monitoring, and forecasting the capabilities of AI systems to conduct AI R&D of their own. While our benchmark does not capture every aspect of real-world research, we believe it marks a substantive step towards rigorous evaluation of AI autonomy in ML research.

Impact Statement

As AI systems progress toward autonomously conducting complex ML research, they offer promise for accelerating scientific discovery in multiple fields. As one pertinent example, AI-driven ML research could significantly accelerate AI safety and alignment research efforts. Being able to replicate cutting-edge ML research from scratch is indicative of an AI system’s autonomy and ML expertise, suggesting that a model capable of high performance on PaperBench would have a non-trivial capacity to tackle real-world, open-ended ML research tasks.

However, the capability to autonomously replicate and ex-

tend frontier research can also lead to rapid innovation that outpaces our ability to fully understand its implications. If powerful models can not only *replicate* state-of-the-art techniques but also iteratively *refine and improve* them, they might accelerate the development of increasingly capable systems at a pace that poses heightened risks. We may see models introduced with minimal time for thorough risk assessment, governance measures, or safety and alignment interventions, potentially leading to hazardous or destabilizing outcomes.

By open-sourcing PaperBench, we aim to provide a method to measure these emerging autonomous R&D capabilities of frontier AI systems. We acknowledge that PaperBench represents just one piece of a broader evaluation landscape for autonomous AI R&D. We encourage future work in anticipating and preparing for the powerful impacts that AI systems with greater autonomy may eventually unlock.

Acknowledgements

We express our sincere gratitude to Arvind Ramaswami and Francisco Garcia for their contributions to the creation of the PaperBench dataset.

We would also like to thank the authors of the papers included in PaperBench for working closely with us to validate each rubric: Alexander Spangher, Ananye Agarwal, Andrew Lee, Bartłomiej Cupiał, Bowen Zhao, Chang Xu, Christian Schlarman, Diana Cai, Dong Gong, Feng Liu, Haotian Sun, Jia Shi, Kevin Frans, Louis Sharrock, Manuel Gloeckler, Michael Samuel Albergo, Pratik Rathore, Shuaicheng Niu, Tim Knappe, Tongliang Liu, Xinyu Xing, and Xisen Jin. Their willingness to clarify methodologies, share additional materials, and support the rubric-writing process was invaluable in creating comprehensive and accurate rubrics for their work.

We also express our thanks to Amy Lu, Anit Kumar Sahu, Armin Wasicek, Brendan Rappazzo, Chun-Wei Chiang, David Khachaturov, Marc Harary, and Stephen Giguere who participated in the human baseline experiments.

Finally, we would like to thank Aparna Dutta, Jerry Tworek, Phoebe Thacker, Phillip Guo, Kevin Liu, Vichyr Pong, Mengyuan Yan and Yining Chen for their advice and collaboration.

References

Albergo, M. S., Goldstein, M., Boffi, N. M., Ranganath, R., and Vanden-Eijnden, E. Stochastic Interpolants with Data-Dependent Couplings. June 2024. URL <https://openreview.net/forum?id=FFILRGD0jG>.

- Anthropic. Responsible Scaling Policy. <https://www.anthropic.com/news/announcing-our-updated-responsible-scaling-policy>, 2024. Accessed: 2025-01-29.
- Cai, C., Ye, Z., Feng, L., Qi, J., and Liu, F. Sample-specific Masks for Visual Reprogramming-based Prompting. June 2024a. URL <https://openreview.net/forum?id=4sikyurTLX>.
- Cai, D., Modi, C., Pillaud-Vivien, L., Margossian, C., Gower, R. M., Blei, D., and Saul, L. K. Batch and match: black-box variational inference with a score-based divergence. June 2024b. URL <https://openreview.net/forum?id=bplNmU2ROC>.
- Chan, J. S., Chowdhury, N., Jaffe, O., Aung, J., Sherburn, D., Mays, E., Starace, G., Liu, K., Maksin, L., Patwardhan, T., et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024.
- Chen, D., Chen, R., Zhang, S., Liu, Y., Wang, Y., Zhou, H., Zhang, Q., Wan, Y., Zhou, P., and Sun, L. MLLM-as-a-Judge: Assessing Multimodal LLM-as-a-Judge with Vision-Language Benchmark. *arXiv preprint arXiv:2402.04788*, 2024.
- Cheng, Z., Wu, X., Yu, J., Yang, S., Wang, G., and Xing, X. RICE: Breaking Through the Training Bottlenecks of Reinforcement Learning with Explanation.
- Chiang, C.-H. and yi Lee, H. Can Large Language Models Be an Alternative to Human Evaluations?, 2023. URL <https://arxiv.org/abs/2305.01937>.
- DeepMind. Specification Gaming: The Flip Side of AI Ingenuity, 2024. URL <https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>. Accessed: 10 March 2025.
- Frans, K., Park, S., Abbeel, P., and Levine, S. Unsupervised Zero-Shot Reinforcement Learning via Functional Reward Encodings. June 2024. URL <https://openreview.net/forum?id=a6wCNfIj8E>.
- Fu, J., Ng, S.-K., Jiang, Z., and Liu, P. GPTScore: Evaluate as You Desire. *arXiv preprint arXiv:2302.04166*, 2023.
- Gloeckler, M., Deistler, M., Weilbach, C. D., Wood, F., and Macke, J. H. All-in-one simulation-based inference. June 2024. URL <https://openreview.net/forum?id=DL79HYCFFq>.
- Google DeepMind. Frontier Safety Framework, May 2024.
- Harvey Team. Introducing BigLaw Bench, August 2024. URL <https://www.harvey.ai/blog/introducing-biglaw-bench>.
- Huang, Q., Vora, J., Liang, P., and Leskovec, J. MLA-gentBench: Evaluating Language Agents on Machine Learning Experimentation. In *Forty-first International Conference on Machine Learning*, 2024.
- Jansen, P., Côté, M.-A., Khot, T., Bransom, E., Mishra, B. D., Majumder, B. P., Tafjord, O., and Clark, P. DISCOVERYWORLD: A Virtual Environment for Developing and Evaluating Automated Scientific Discovery Agents. *arXiv preprint arXiv:2406.06769*, 2024.
- Jin, X. and Ren, X. What Will My Model Forget? Forecasting Forgotten Examples in Language Model Refinement. June 2024. URL <https://openreview.net/forum?id=bzNwexOPWm>.
- Jing, L., Huang, Z., Wang, X., Yao, W., Yu, W., Ma, K., Zhang, H., Du, X., and Yu, D. DSbench: How Far Are Data Science Agents to Becoming Data Science Experts? *arXiv preprint arXiv:2409.07703*, 2024.
- Lambert, N., Pyatkin, V., Morrison, J., Miranda, L., Lin, B. Y., Chandu, K., Dziri, N., Kumar, S., Zick, T., Choi, Y., Smith, N. A., and Hajishirzi, H. RewardBench: Evaluating Reward Models for Language Modeling, 2024. URL <https://arxiv.org/abs/2403.13787>.
- Lee, A., Bai, X., Pres, I., Wattenberg, M., Kummerfeld, J. K., and Mihalcea, R. A Mechanistic Understanding of Alignment Algorithms: A Case Study on DPO and Toxicity. June 2024. URL <https://openreview.net/forum?id=dBqHGZPGZI>.
- Niu, S., Miao, C., Chen, G., Wu, P., and Zhao, P. Test-Time Model Adaptation with Only Forward Passes. June 2024. URL <https://openreview.net/forum?id=qz1Vxlv9iK>.
- OpenAI. Preparedness Framework, December 2023.
- Pan, A., Bhatia, K., and Steinhardt, J. The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models, 2022. URL <https://arxiv.org/abs/2201.03544>.
- Rathore, P., Lei, W., Frangella, Z., Lu, L., and Udell, M. Challenges in Training PINNs: A Loss Landscape Perspective. June 2024. URL <https://openreview.net/forum?id=mJGiFr8jLa>.
- Sanchez, G., Spangher, A., Fan, H., Levi, E., and Biderman, S. Stay on Topic with Classifier-Free Guidance. June 2024. URL <https://openreview.net/forum?id=RiM3cl9MdK>.

- Sawada, T., Paleka, D., Havrilla, A., Tadepalli, P., Vidas, P., Kranias, A., Nay, J. J., Gupta, K., and Komatsuzaki, A. ARB: Advanced Reasoning Benchmark for Large Language Models, 2023. URL <https://arxiv.org/abs/2307.13692>.
- Schlarman, C., Singh, N. D., Croce, F., and Hein, M. Robust CLIP: Unsupervised Adversarial Fine-Tuning of Vision Embeddings for Robust Large Vision-Language Models. June 2024. URL <https://openreview.net/forum?id=WLPPhywf1si>.
- Sharrock, L., Simons, J., Liu, S., and Beaumont, M. Sequential Neural Score Estimation: Likelihood-Free Inference with Conditional Score Based Diffusion Models. June 2024. URL <https://openreview.net/forum?id=8viuf9PdZU>.
- Shi, J., Gare, G. R., Tian, J., Chai, S., Lin, Z., Vasudevan, A. B., Feng, D., Ferroni, F., and Kong, S. LCA-on-the-Line: Benchmarking Out of Distribution Generalization with Class Taxonomies. June 2024. URL <https://openreview.net/forum?id=HPXRzM9BYZ>.
- Si, C., Yang, D., and Hashimoto, T. Can LLMs Generate Novel Research Ideas? A Large-Scale Human Study with 100+ NLP Researchers. *arXiv preprint arXiv:2409.04109*, 2024.
- Siegel, Z. S., Kapoor, S., Nagdir, N., Stroebl, B., and Narayanan, A. CORE-Bench: Fostering the Credibility of Published Research Through a Computational Reproducibility Agent Benchmark. *arXiv preprint arXiv:2409.11363*, 2024.
- Singla, J., Agarwal, A., and Pathak, D. SAPG: Split and Aggregate Policy Gradients. June 2024. URL <https://openreview.net/forum?id=4dOJAfXhNV>.
- Sun, H., Zhuang, Y., Wei, W., Zhang, C., and Dai, B. BBox-Adapter: Lightweight Adapting for Black-Box Large Language Models.
- UK AI Safety Institute. Inspect, 2025. URL <https://inspect.ai-safety-institute.org.uk/agents.html#sec-basic-agent>.
- van der Weij, T., Hofstätter, F., Jaffe, O., Brown, S. F., and Ward, F. R. AI Sandbagging: Language Models Can Strategically Underperform on Evaluations. *arXiv preprint arXiv:2406.07358*, 2024.
- Wang, R., Jansen, P., Côté, M.-A., and Ammanabrolu, P. Scienceworld: Is Your Agent Smarter than a 5th Grader? *arXiv preprint arXiv:2203.07540*, 2022.
- Wang, T., Kulikov, I., Golovneva, O., Yu, P., Yuan, W., Dwivedi-Yu, J., Pang, R. Y., Fazel-Zarandi, M., Weston, J., and Li, X. Self-Taught Evaluators, 2024a. URL <http://arxiv.org/abs/2408.02666>.
- Wang, X., Lin, B., Liu, D., Chen, Y.-C., and Xu, C. Bridging Data Gaps in Diffusion Models with Adversarial Noise-Based Transfer Learning. June 2024b. URL <https://openreview.net/forum?id=PpBs2iL0jv>.
- Wijk, H., Lin, T., Becker, J., Jawhar, S., Parikh, N., Broadley, T., Chan, L., Chen, M., Clymer, J., Dhyani, J., et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024.
- Wolczyk, M., Cupial, B., Ostaszewski, M., Bortkiewicz, M., Zajac, M., Pascanu, R., Kucinski, L., and Milos, P. Fine-tuning Reinforcement Learning Models is Secretly a Forgetting Mitigation Problem. June 2024. URL <https://openreview.net/forum?id=53iSXblm8w>.
- Xia, X., Liu, J., Zhang, S., Wu, Q., Wei, H., and Liu, T. Refined Coreset Selection: Towards Minimal Coreset Size under Model Performance Constraints. June 2024. URL <https://openreview.net/forum?id=yb5xV8LFDq>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models, March 2023. URL <http://arxiv.org/abs/2210.03629>. arXiv:2210.03629 [cs].
- Zhao, B., Hajishirzi, H., and Cao, Q. APT: Adaptive Pruning and Tuning Pretrained Language Models for Efficient Training and Inference. June 2024. URL <https://openreview.net/forum?id=sb81Xl50JG>.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36: 46595–46623, 2023.
- Zhuge, M., Zhao, C., Ashley, D., Wang, W., Khizbullin, D., Xiong, Y., Liu, Z., Chang, E., Krishnamoorthi, R., Tian, Y., et al. Agent-as-a-Judge: Evaluate Agents with Agents. *arXiv preprint arXiv:2410.10934*, 2024.

A. Future Directions in AI Evaluation

Our experience working on PaperBench has driven home important lessons about the future of AI development and evaluation. On many tasks (outside of PaperBench), AI systems currently perform near or at the level of expert humans, and there is demand to offload to AI systems tasks that are labor-intensive for humans.

Unlike most present evaluations, PaperBench expects the evaluated agents to produce a complex and unstructured output. Additionally, the output cannot be programmatically graded. Rubrics (Sawada et al., 2023; Harvey Team, 2024) offer one approach for converting the complex and underspecified task into something simpler and more well-specified: To deal with complexity, we break evaluation into smaller sub-criteria. To deal with underspecification, we collaborate with authors from the papers to make specific choices about what is important for replication and how to weigh the importance of various factors (there exist many different realizations of our paper rubrics which are no less valid).

Nevertheless, important limitations of PaperBench remain (see Section 7). Below, we discuss directions that would help solve those limitations and unlock scalable yet trustworthy evaluation of complex and unstructured tasks more generally.

A.1. Exploring rubric design and creation

In the rubrics designed for PaperBench, the order of child nodes encodes the dependencies; later child nodes are dependent on earlier child nodes. For example, requirements for matching results from the paper come after requirements for implementing the methodology required for running the experiments. However, our current design doesn’t specify exactly which of the previous requirements are necessary; ideally, the design of the rubric would specify which requirements are necessary for any given requirement in the rubric. Future work could explore using dependency graphs in the rubrics as a potential solution.

Given the difficulty of creating rubrics, automated rubric creation is another valuable direction for future work. Our preliminary experiments found that frontier models, such as o1, are excellent partners for understanding and summarizing papers. However, we found frontier models struggle to create reliable rubrics from start to end, even with significant prompt engineering. Due to rubric complexity, it is also a challenge to review and iterate on model-generated rubrics, but we believe human-in-the-loop workflows might prove fruitful here. We also believe that using models to critique rubrics is a promising approach that could lead to faster rubric creation.

A.2. Improving automated judges

A single rubric in PaperBench typically has hundreds of nodes to evaluate, which is prohibitively expensive to evaluate with humans; in preliminary experiments, we found grading using expert humans to take on the order of tens of hours. As demonstrated by SimpleJudge (Section D), model-based evaluations (Chiang & yi Lee, 2023; Lambert et al., 2024; Wang et al., 2024a) play an important role in the scalability of rubric-based evaluation. We’ve taken an early step with JudgeEval (Section 4.2) to measure the accuracy of model-based judges on tasks with huge complex outputs, but more work is needed to improve accuracy and understand their strengths and weaknesses.

We further note that the more reliable your judge is, the less fine-grained your rubric’s task decomposition needs to be, potentially reducing the effort necessary for task decomposition as judges become more capable. We leave it to future work to study the trade-off between careful specification and delegation to the judge.

A.3. Specification gaming and adversarial agents

PaperBench rubrics have been carefully designed to avoid false negatives and false positives, but given the large number of nodes and the complexity of paper replication, we cannot yet rule out loopholes in our evaluation. Agents may have incentives to strategically underperform (van der Weij et al., 2024) or overperform (DeepMind, 2024; Pan et al., 2022) on PaperBench; future work could explore both the capability and propensity of agents to convincingly underperform and overperform on PaperBench.

B. Paper Selection Process

Our final dataset was constructed through a systematic filtering process applied to papers from ICML 2024. Each filter was designed to ensure papers in our dataset would be suitable for replication attempts. We used gpt-4o-2024-08-06 with a series

of prompts to implement initial filtering. Below we detail each filtering step and its rationale:

- **Commercial and Geographic Filter:** Papers were excluded if 75% or more of the authors had affiliations suggesting it would be unlikely for the authors to collaborate, due to constraints involving working with commercial labs or certain countries.
- **Empirical Content Filter:** At least one of the contributions from the paper must involve a substantial empirical experiment, which requires non-trivial engineering to replicate. This rules out position papers and pure theory papers. We also rule out papers that primarily present a new software framework, library, or tool since they do not present novel experimental results.
- **Hardware Requirements Filter:** Papers requiring distributed training across multiple compute nodes were excluded. This ensures that all remaining papers can be reproduced on a single machine, making replication more accessible.
- **Model Dependency Filter:** Papers depending on closed-source pretrained models (e.g., GPT-4, Claude, PaLM) were excluded.
- **Data Requirements Filter:** Papers requiring human data collection or annotation were excluded. This ensures reproducibility without the need for new human participants or annotators.
- **Reproducibility Filter:** Sufficient detail must be present in the paper such that it is possible to replicate the results from scratch by reading it and following the methodology. See Section 3.2 for further discussion of how we ensure that papers are replicable even when lacking some information.
- **Framework Papers Filter:** Papers primarily introducing new software frameworks or libraries were excluded, as these typically require different replication approaches than research papers.
- **Accessible Dependencies Filter:** All the paper’s dependencies should be easily accessible. If a paper has dependencies that are inaccessible (e.g., modifying closed-source models) or are fast-to-change (e.g., an unreliable API endpoint that frequently changes), these must be substitutable or able to be dropped without making the remaining paper replication uninteresting or impossible.

We then randomly selected the remaining Spotlight and Oral papers and read the paper to ensure that there were no remaining issues that the automated filtering had missed. We reached out to authors of papers that passed this review process until we had reached out to 42 authors and had secured 20 authors who agreed to work with us to produce rubrics for their papers.

C. Rubric and Addendum Creation Process

Each rubric is collaboratively developed with one of the original authors of the paper to ensure accuracy and relevance. The process begins with two research engineers drafting the initial rubric, which undergoes several rounds of internal review to refine its structure and content.

Once the internal review is complete, the rubric is shared with the original author, who works with us under a formal agreement to verify its correctness and provide expert input. This phase often involves multiple rounds of feedback to address ambiguities or questions about the paper’s methods or results. Any clarifications from the author are incorporated into the paper’s addendum.

On average, the creation of a rubric and its addendum takes many tens of hours of labor. We refer to Figure 4 for an excerpt from one of the rubrics in PaperBench.

D. SimpleJudge Implementation

Given a submission, our judge independently grades each leaf node in a rubric. For a specific leaf node, the judge is prompted with the Markdown of the paper, the addendums, prior requirements in the rubric (siblings and direct ancestors), the leaf node’s requirement, and relevant files from the submission.

We collect the relevant files of the submission for context-management reasons and do so as follows:

Table 7. Counts of Nodes and Leaf Nodes in Rubrics

Rubric	Total Nodes	Leaf Nodes	Code Dev.	Execution	Res. Match
adaptive-pruning	172	123	86	10	27
all-in-one	234	174	92	62	20
bam	1021	789	255	518	16
bbox	422	279	145	81	53
bridging-data-gaps	207	172	55	46	71
fre	636	437	306	124	7
ftrl	233	178	120	20	38
lbcs	1471	916	485	410	21
lca-on-the-line	1048	819	403	370	46
mechanistic-understanding	128	96	36	44	16
pinn	2551	1963	126	1815	22
rice	489	361	178	170	13
robust-clip	146	106	70	8	28
sample-specific-masks	396	331	87	223	21
sapg	279	206	77	64	65
sequential-neural-score-estimation	123	92	67	5	20
stay-on-topic-with-classifier-free-guidance	186	121	70	35	16
stochastic-interpolants	94	69	58	7	4
test-time-model-adaptation	236	163	86	36	41
what-will-my-model-forget	1146	921	872	28	21

For `Code Development` and `Execution` leaf nodes, the submission directory is first filtered via simple whitelisting of source code, documentation and configuration files (e.g. markdown, python, JSON, TOML, C++, etc.), and blacklisting of anything originating from directories not related to source code, e.g. `venv` directories. If the entirety of the filtered submission fits within $(n_{ctx} - 10,000)$, where n_{ctx} is the size of the context window of the underlying model used in the judge, then all files are concatenated (with filenames added to the top of each file) and added to the context window. Otherwise, we further filter the codebase by having the judge rank the files and only include the top ten in its context. We display all filenames to the judge, ask it to rank the files in order of relevance to the current requirement, then add files in descending order of relevance until the context window limit of $(n_{ctx} - 10,000)$ tokens would be exceeded.

For `Result Match` nodes, we follow the same exact process, but rather than whitelisting source code and configuration files, we whitelist plaintext files likely to contain tabular data (such as CSV, JSON, JSONL, HTML, etc.) that have a last-modified timestamp that is newer than the start-time of the `reproduce.sh` execution. Just like in `Code Development` and `Execution`, we whitelist documentation files and blacklist non-source directories.

See Figure 7 for the prompt used for file ranking.

Having identified the relevant files, we prompt our judge to assess whether the requirement of the leaf node has been fulfilled and to provide the reasoning for the binary score it chose; see Figures 8 and 9 for the prompt we use. We then use `gpt-4o`²⁰ to parse the response of the judge model for a score that should be 0 or 1, an explanation that should be a short summary of the judge model’s reasoning, and `valid_score` – a boolean indicating whether the response contained a valid score.

E. Monitor Implementation

Our basic monitor implementation performs a simple text search on log files to identify occurrences of blacklisted terms. When a blacklisted term is found, the monitor logs the specific term and a few lines of surrounding context to aid human review.

²⁰`gpt-4o-2024-08-06`

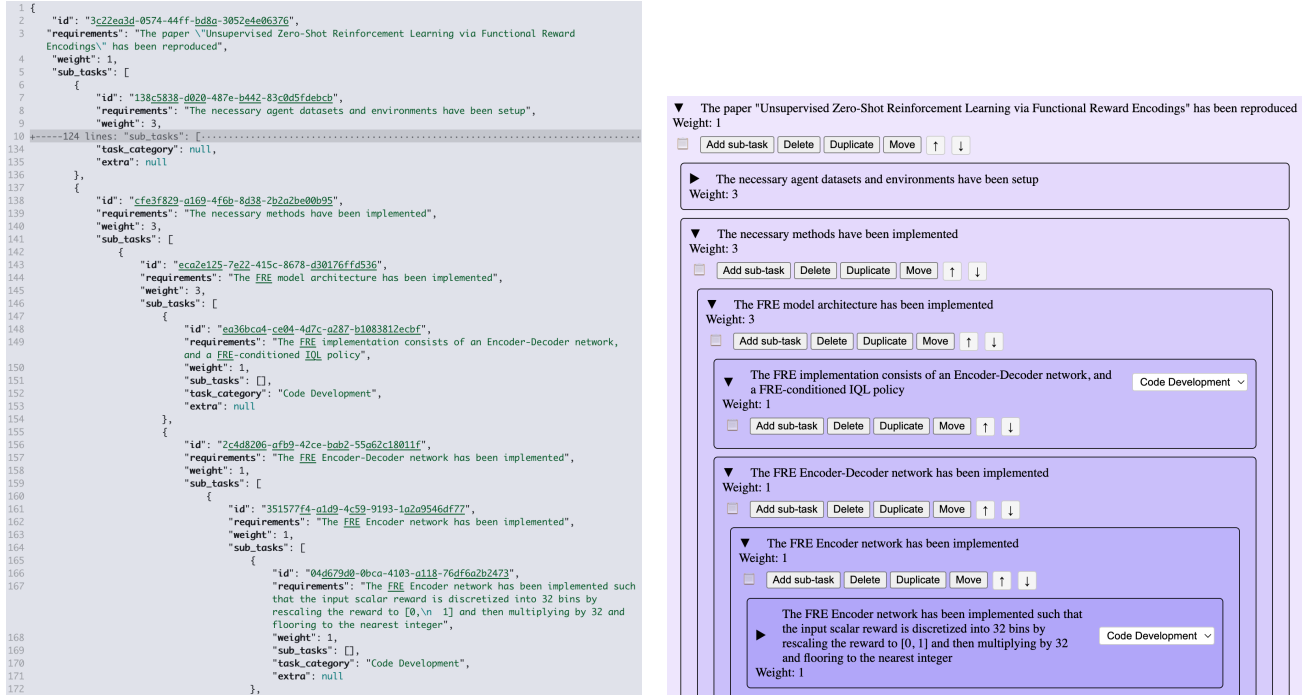


Figure 4. An excerpt of the rubric for one of the papers in PaperBench. Shown in the underlying JSON (left) and in a GUI (right).

F. Agent Implementation

When executing agents with a time limit, we did not count time spent retrying when querying the model API towards the time limit; these retries were commonly due to rate limits or server errors.

F.1. BasicAgent

Our agent scaffold is a basic ReAct (Yao et al., 2023) agent architecture that runs a tool use loop until the agent runs out of steps or time. It is based on Inspect AI’s basic agent (UK AI Safety Institute, 2025) with the following changes:

- We re-frame the submit tool as an “end task” tool that the agent should call when it is completely finished with its attempt, in order to dissuade the agent from calling it too soon.
- We add a simple context length management system that removes old non-instruction messages from the agent’s context when the context limit is approached.
- We provide a paginated file reader tool that allows the agent to read a file in a piecemeal fashion and also search a file for keywords.

We devise our system prompt used by starting with the original Basic Agent system prompt and adjusting it based on the results of preliminary experiments which used a small subset of our final dataset. In these preliminary experiments we found various failure modes such as:

- Models simply describe a plan for how to replicate the paper instead of calling any tools to write code and make a replication itself.
- Reasoning models such as o1 were particularly prone to trying to finish the task in a single response, calling the submit tool with a huge message containing multiple pieces of code.
- Models didn’t attempt to read the full paper, and so naturally weren’t able to complete a full replication.